

EEE F435: Digital Image Processing

Assignment 2

Pranav Chandra N. V.
2023AAPS0013P

29th September, 2025

Contents

1	Introduction	3
2	Task 1	3
2.1	Image Frequency Domain Display	3
2.2	Swapping Magnitude and Phase	3
2.3	Shifting of Image	4
2.3.1	Circular Shifting	4
2.3.2	Translation of Image	5
2.4	Scaling of Image	5
2.5	Rotation of Image	6
3	Task 2 - Filters	8
3.1	Ideal LPF	8
3.2	Butterworth LPF	10
3.3	Gaussian LPF	11
3.4	Ideal HPF	12
3.5	Butterworth HPF	13
3.6	Gaussian HPF	14
3.7	Adding and Removing Noise	14
4	Conclusion	16

List of Figures

1	Images in Time and Frequency Domain	4
2	Images 1 and 2 with swapped magnitudes	4
3	Circularly Shifted Images with Magnitude and Phase	5
4	Scaled Image with Magnitude and Phase	6
5	Image rotated by various angles	7
6	Circular LPF of Various Sizes	9
7	Rectangular LPF of Various Sizes	9
8	Butterworth Filtered Images of Various Orders and Sizes	10
9	Gaussian Filtered Images of Various Sizes	11
10	Ideal Circular HPF of Various Sizes	12
11	Ideal Rectangular HPF of Various Sizes	12
12	Butterworth HPF of Various Orders and Sizes	13
13	Gaussian HPF of Various Sizes	14
14	Original Image 1, Noised, Cleaned with Ideal LPF, Cleaned with Butterworth LPF	15
15	Original Image 2, Noised, Cleaned with Ideal LPF, Cleaned with Butterworth LPF	15

1 Introduction

The goal of this assignment was to work with the functions that are used for operations in the frequency domain. All code was done in MATLAB, some of this code is provided in the appendix, with snippets provided within the main body to maintain brevity and flow. Two images were used, one from the first assignment, and the second taken recently for this assignment. In some cases, both images have been used, while in others, only one has been used (for simplicity's sake).

All images were converted to grayscale, and both images were resized to the same size to allow all operations (especially in Task 2.2). The entire code base can be found appended to the end of this report, following the guidelines provided in the assignment brief.

2 Task 1

This part was basic frequency domain operations on images, using the fast fourier transform operation within MATLAB.

2.1 Image Frequency Domain Display

Both sample images have been imported and shown here. In order to display the frequency domain aspects, the `fft` class of commands was used. In all cases, `fft2` was used, as we're working with 2D images here. Figure 1

```
1      fft_1 = fft2(img_1);
2      fftshift_1 = fftshift(fft_1);
3      mag_1 = log(1 + abs(fftshift_1));
4      phase_1 = angle(fftshift_1);
```

2.2 Swapping Magnitude and Phase

When swapping the magnitude and phase of the two images, the most important consideration was to make sure that both the images were the same size. To do this, the `imresize` command was used.

To swap the magnitudes and phase, the $a + jb = |M|e^{angle}$ property was used with the following code.

```
1      mixed_1 = mag_1 .* exp(1i * phase_2);
2      mixed_1 = ifft2(ifftshift(mixed_1));
3      mixed_2 = mag_2 .* exp(1i * phase_1);
4      mixed_2 = ifft2(ifftshift(mixed_2));
```

The resulting images are shown in Figure 2.

It can be observed that the image with the phase of image 1 and magnitude of image 2 looks more like image 1, while the other image looks more like image 2. This shows that the phase component of an image is more important than the magnitude component.

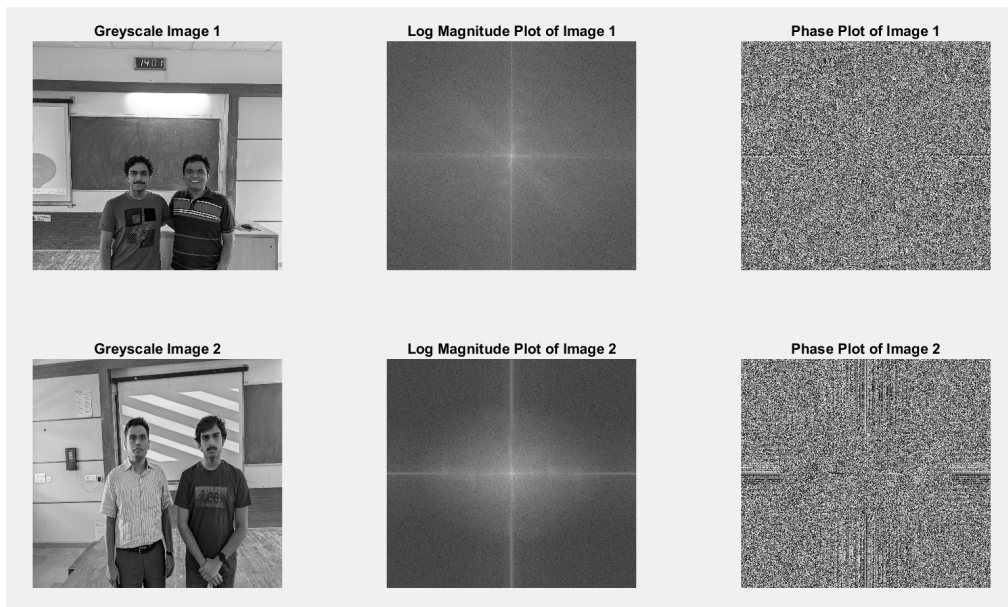


Figure 1: Images in Time and Frequency Domain

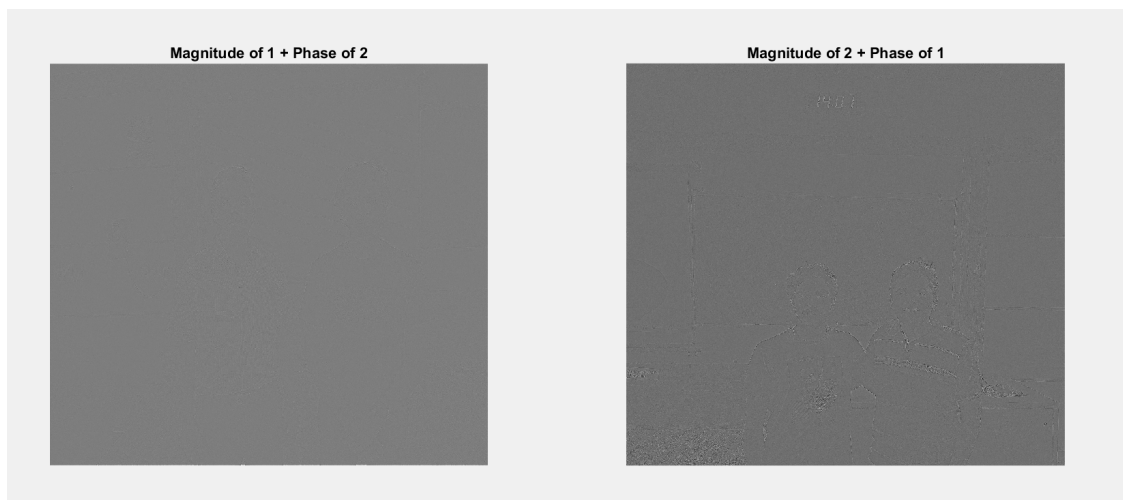


Figure 2: Images 1 and 2 with swapped magnitudes

2.3 Shifting of Image

Here, we had to shift the image. Shifting the image can be done in two ways. Using the `circshift` command, or using the `imtranslate` command. There is a stark difference in how these commands work, so the explanation has been divided into the subsequent two subsections.

2.3.1 Circular Shifting

The first tried test was circular shifting. The image was shifted to the right and down by the appropriate amounts, and the parts that were shifted out simply got shifted to the top and left of the image.

```

1 shift_x = A * 10 + 5;
2 shift_y = B * 10 + 5;
3 img_1_shifted = circshift(img_1, [shift_y, shift_x]);
4 img_2_shifted = circshift(img_2, [shift_y, shift_x]);

```

The resulting images are shown in Figure 3. Unfortunately, the shifting is very small

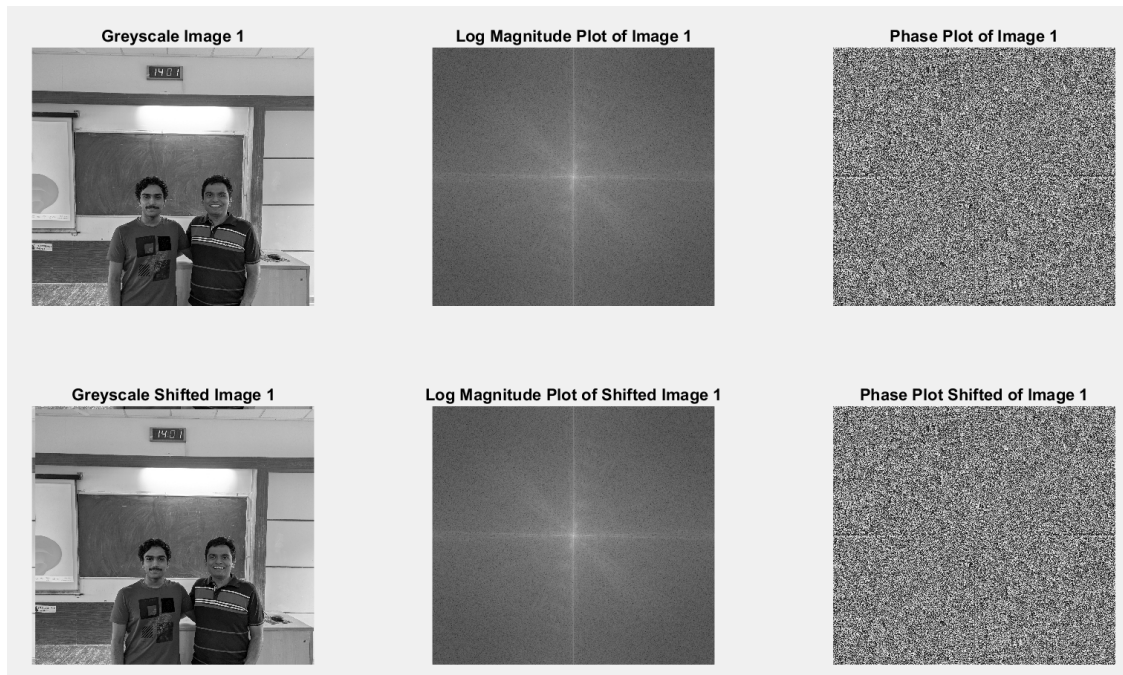


Figure 3: Circularly Shifted Images with Magnitude and Phase

so not very visible unless you zoom in to the corners. However, in the frequency domain, an important fact emerges. Circular shifting causes no change in the frequency domain representation of the image. This is because circular shifting is equivalent to a periodic extension of the image, which does not affect its frequency components.

2.3.2 Translation of Image

The second method used was translation. This is done using the `imtranslate` command. This command shifts the image, and fills in the parts that are shifted out with a default value (0, black).

2.4 Scaling of Image

This task required us to scale the image. For brevity, only one image was scaled in accordance with the guidelines. The `imresize` command was used to perform this modification.

Figure 4 shows the scaled image, along with its magnitude plot. It can be seen that the scaling causes a change in the magnitude plot. However, the scale down

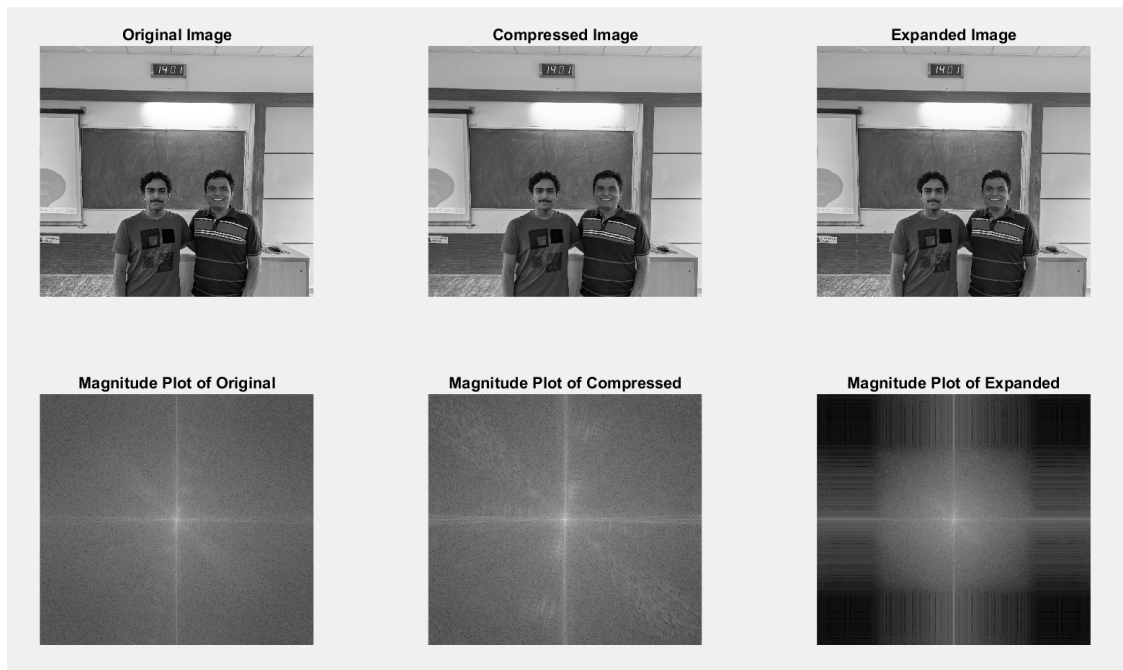


Figure 4: Scaled Image with Magnitude and Phase

effect cannot be seen at this resolution, but the effect of scale up on magnitude is very clear.

|||

2.5 Rotation of Image

For the rotation of the image, the `imrotate` command was used. This command rotates the image by a specified angle, and fills in the parts that are rotated out with a default value (0, black). A simple for loop was used to generate the images at various angles, shown below.

```

1 angles = [0, 30, 45, 60, 90];
2 figure();
3 for k = 1:length(angles)
4     theta = angles(k);
5     img_rot = imrotate(img2, theta, 'bilinear', 'crop');
6
7     F = fftshift(fft2(img_rot));
8     mag = log(1 + abs(F));
9
10    subplot(2, 5, k);
11    imshow((img_rot));
12    title(['Image after ', num2str(theta), ' rotation']);
13    subplot(2, 5, k+5);
14    imshow(mag, []);
15    title(['Magnitude at ', num2str(theta), ' rotation']);
16 end

```

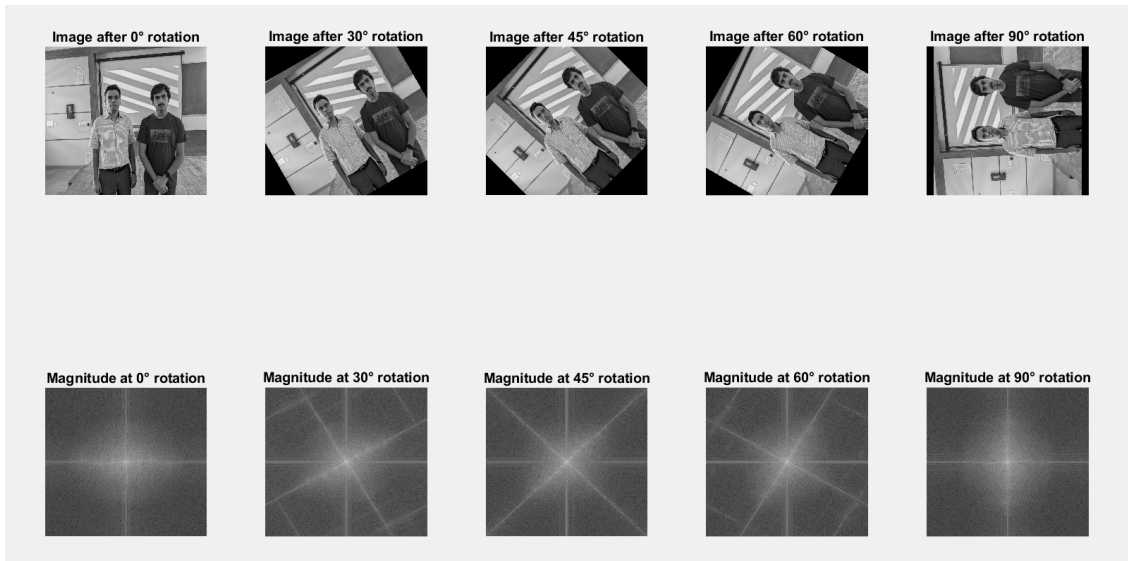


Figure 5: Image rotated by various angles

It can be seen that the rotation in the time domain also causes a rotation in the frequency domain plot.

3 Task 2 - Filters

In this section, we move towards applying a variety of filters on the image. Both LPF and HPF filters are used. For all LPF filters, image 1 was used as a demo, and for all HPF filters, image 2 was used.

3.1 Ideal LPF

The ideal low pass filter is defined as:

$$D(u, v) = \begin{cases} 1 & D(u, v) < R_0 \\ 0 & D(u, v) > R_0 \end{cases} \quad (1)$$

This filter only acts on the frequency domain. To identify the effects of a radial ideal LPF, radii 20, 50 and 100 pixels were chosen, and then applied. Once applied on the frequency domain, the image was converted back to time domain using the `ifft2` command.

To efficiently generate them, a quick for loop was used. Very similar code was used for the rectangular LPF.

```
1 radii = [20, 50, 100];
2 for k = 1:length(radii)
3     H = dist1 <= radii(k); % circular mask
4
5     F1_filt = fft_shifted_1 .* H;
6     img1_filt = real(ifft2(ifftshift(F1_filt)));
7     imgs1{end+1} = img1_filt;
8     F1s{end+1} = F1_filt;
9 end
```

The magnitude and phase plots of these circular LPF images were taken, and shown in Figure 6.

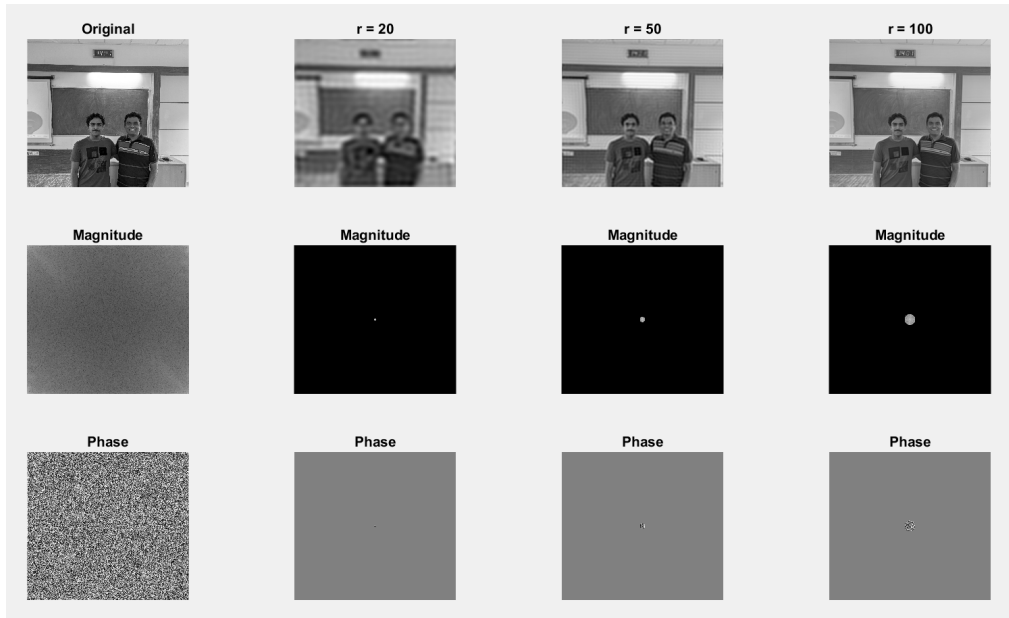


Figure 6: Circular LPF of Various Sizes

Now, for the rectangular LPF, the same process was used. The resultant images can be found in Figure 7.

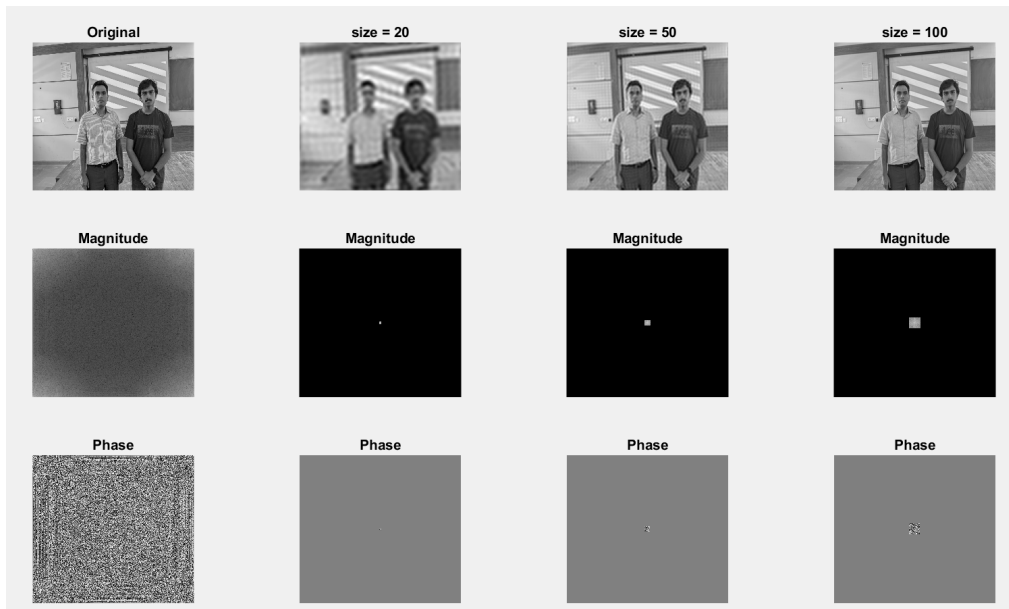


Figure 7: Rectangular LPF of Various Sizes

3.2 Butterworth LPF

The butterworth LPF is mathematically defined as:

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0} \right)^{2n}} \quad (2)$$

Where D_0 is the cutoff frequency, and n is the order of the filter. For this assignment, $n = 1, 2, 4$ was used. The same radii as before were used, and the results are shown in Figure 8.

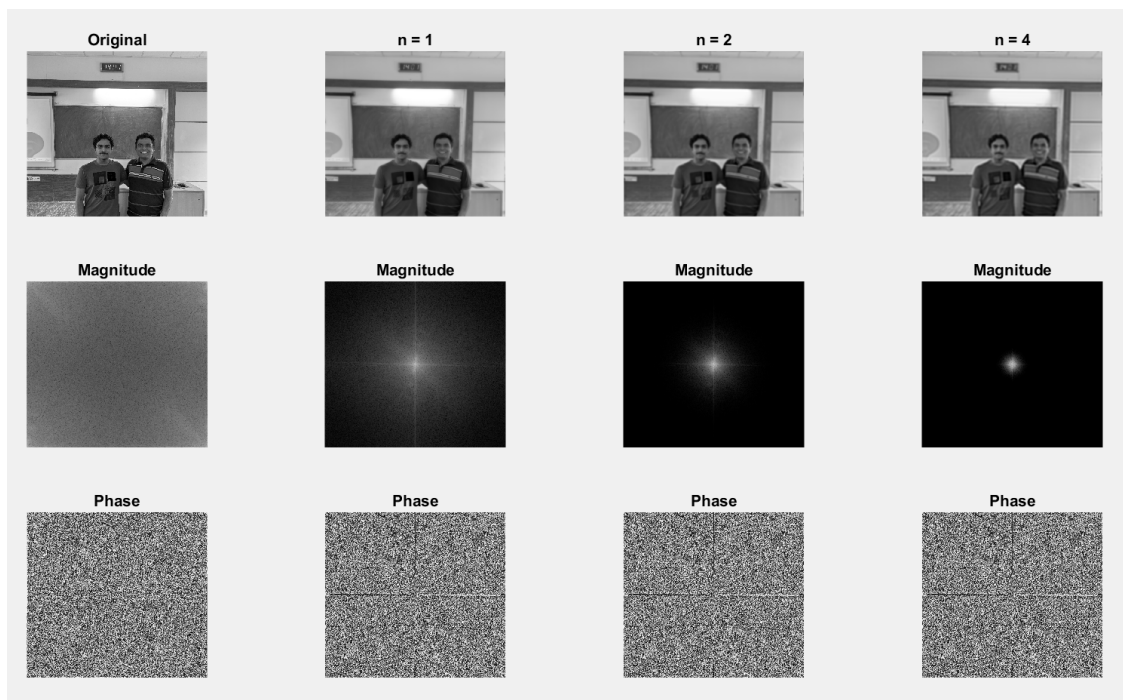


Figure 8: Butterworth Filtered Images of Various Orders and Sizes

The implementation in matlab was very simple, and very similar to the previous code. H was just changed as shown below.

```

1 H = 1 ./ (1 + (D./D0).^(2*n));
2 F_filt = fft_shifted_1 .* H;
3 img_filt = real(ifft2(ifftshift(F_filt)));

```

Higher frequency components from the image were removed as the order of the butterworth filter was increased, this can be seen in both the magnitude graph, and the images, where noisy bands were removed.

3.3 Gaussian LPF

The Gaussian LPF is mathematically defined as:

$$H(u, v) = e^{-\frac{D^2(u, v)}{2D_0^2}} \quad (3)$$

Where D_0 is the cutoff frequency. The same radii as before were used, and the results are shown in Figure 9. The matlab implementation is shown below.

```

1 H = exp(-(D.^2)./(2*(D0^2)));
2 F_filt = fft_shifted_1 .* H;
3 img_filt = real(ifft2(ifftshift(F_filt)));

```

It can be seen that the Gaussian LPF has a very smooth transition, and does not have the ringing effect that the ideal LPF has. This is because the Gaussian function is continuous and smooth, while the ideal LPF has a sharp cutoff.

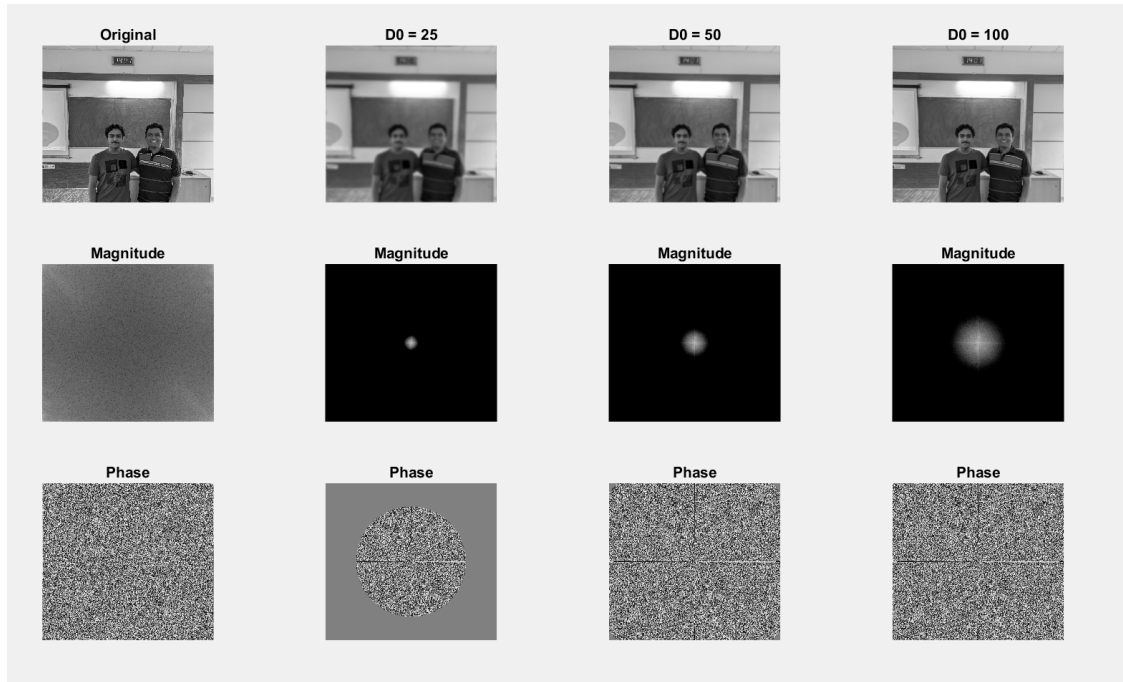


Figure 9: Gaussian Filtered Images of Various Sizes

The smooth nature of the Gaussian Filter can be seen in the magnitude plot of each of the images, where the central circle tapers off smoothly, rather than a sharp cutoff. As the size of the filter was increased, the image came into more focus, and more details were visible. It's likely that a size of about 200 would be sufficient to capture all details while ignoring high frequency noise.

3.4 Ideal HPF

The ideal high pass filter is defined as:

$$D(u, v) = \begin{cases} 0 & D(u, v) < R_0 \\ 1 & D(u, v) > R_0 \end{cases} \quad (4)$$

This filter only acts on the frequency domain, but often produces sharp ringing effects due to the high cutoff.

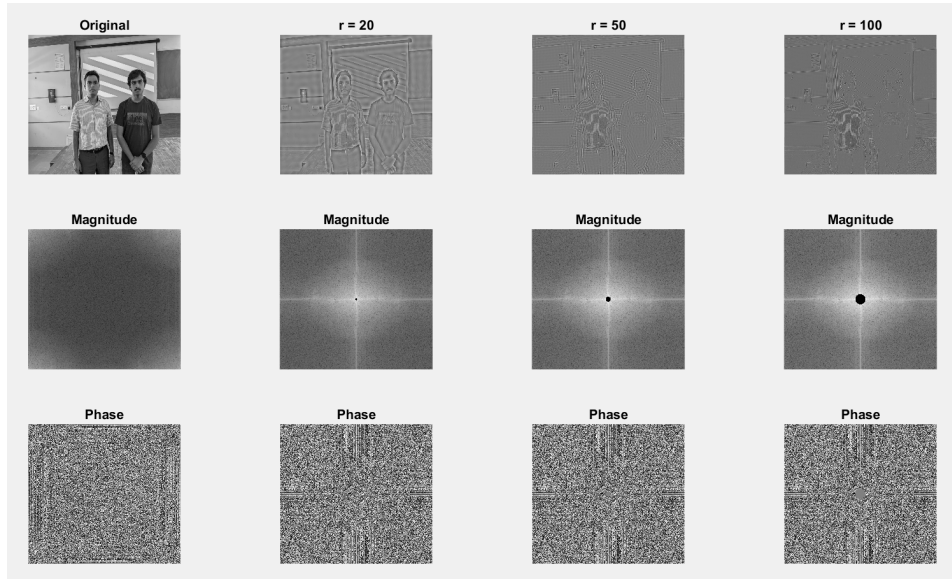


Figure 10: Ideal Circular HPF of Various Sizes

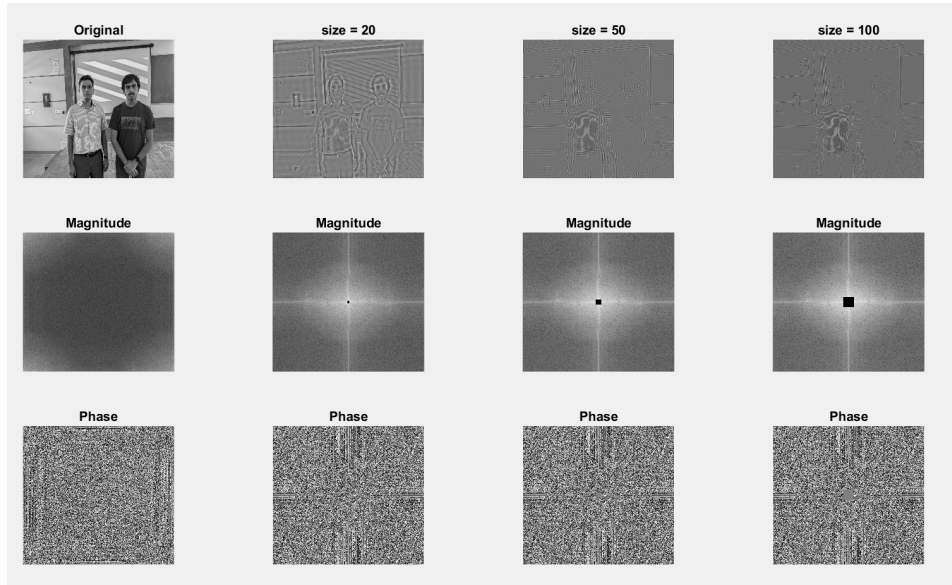


Figure 11: Ideal Rectangular HPF of Various Sizes

Figures 10 and 11 shows the results of applying the ideal HPF with radii 20, 50 and 100 pixels. The ringing effect is very visible in the images, especially at lower radii. As the radius increases, more high frequency components are allowed through, and the image becomes clearer, but the ringing effect remains.

3.5 Butterworth HPF

The butterworth HPF is mathematically defined as:

$$H(u, v) = \frac{1}{1 + \left(\frac{D_0}{D(u, v)} \right)^{2n}} \quad (5)$$

Where D_0 is the cutoff frequency, and n is the order of the filter. For this assignment, $n = 1, 2, 4$ was used. The same radii as before were used, and the results are shown in Figure 12.

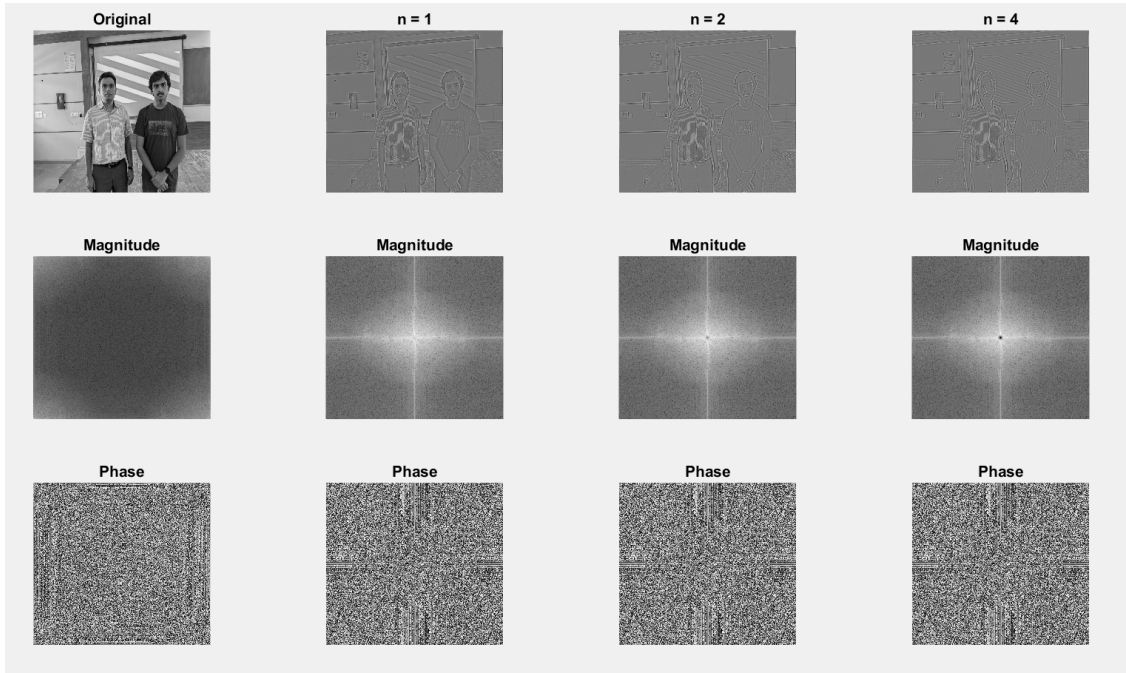


Figure 12: Butterworth HPF of Various Orders and Sizes

The initial image had the most noise, and as the order of the butterworth filter was increased, this noise to signal ratio went down, with signal amount going higher. This is because the main signal components are contained in the lower frequency components. The outline can be seen in these filtered images.

3.6 Gaussian HPF

The Gaussian HPF is mathematically defined as:

$$H(u, v) = 1 - e^{-\frac{D^2(u, v)}{2D_0^2}} \quad (6)$$

Where D_0 is the cutoff frequency. The same radii as before were used, and the results are shown in Figure 13. The matlab implementation is shown below.

```
1 for D0 = cutoffs
2     H = 1 - exp(-(D.^2)./(2*(D0^2)));
3     F_filt = fft_shifted_2 .* H;
4     img_filt = real(ifft2(ifftshift(F_filt)));
5     imgs1{end+1} = img_filt; F1s{end+1} = F_filt;
6 end
```

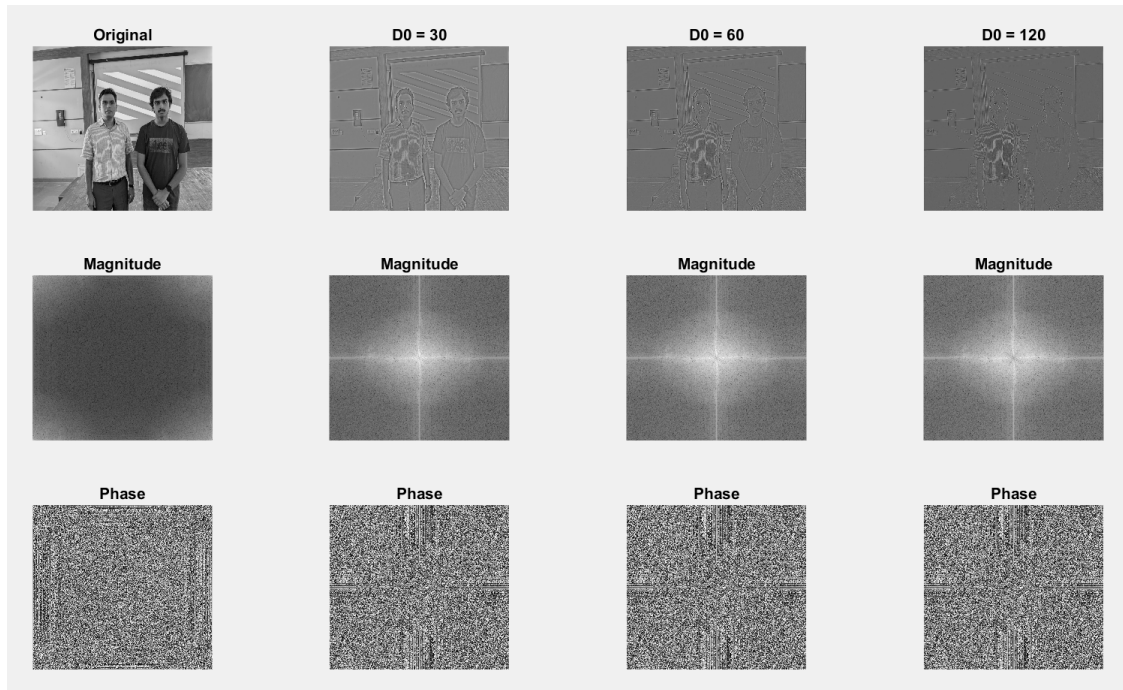


Figure 13: Gaussian HPF of Various Sizes

The Gaussian HPF has a smooth transition, and does not have the ringing effect that the ideal HPF has. As the size of the filter was increased, the features of the image kept reducing as expected.

3.7 Adding and Removing Noise

The final task of this assignment was to add noise to an image, and then remove it using filtering. The following noise profile was added to the image:

```
1 noise = noise_amp * sin(2*pi * f_noise * xgrid / N);
2 I_noisy = I + noise;
```

This noise addition and removal was done on both the images.

In order to really show the difference between the filters, both an ideal LPF and a Butterworth LPF were used. A comparison can be seen on the images.

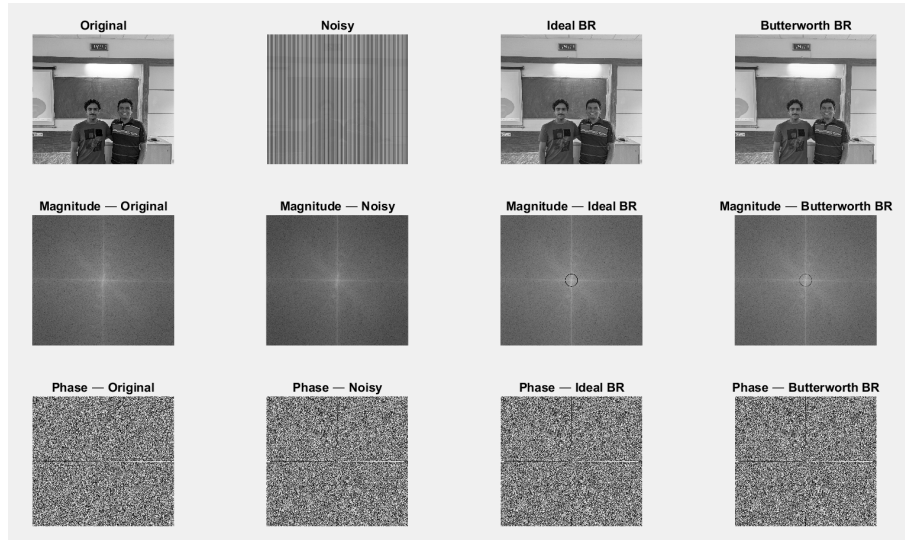


Figure 14: Original Image 1, Noised, Cleaned with Ideal LPF, Cleaned with Butterworth LPF

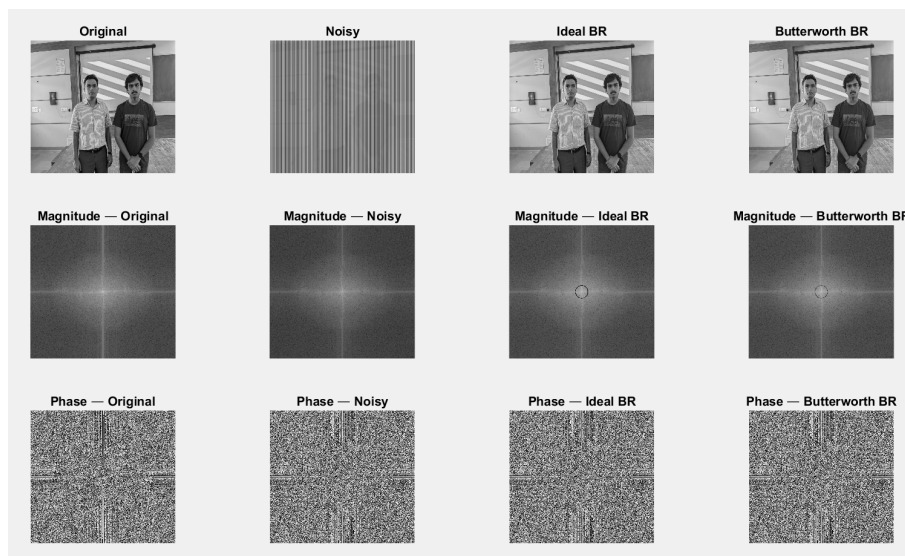


Figure 15: Original Image 2, Noised, Cleaned with Ideal LPF, Cleaned with Butterworth LPF

Figures 14 and 15 show the original image, the noisy image, the image cleaned with an ideal LPF, and the image cleaned with a Butterworth LPF. It can be seen that both filters are able to remove the noise, but the Butterworth LPF does a better job of preserving the details of the image while removing the noise. The ideal LPF, while effective at removing noise, also removes some of the finer details of the image, leading to a loss of quality.

4 Conclusion

This assignment explored key concepts of frequency-domain image processing using MATLAB:

- **Importance of Phase vs. Magnitude:**

- Phase $\angle F(u, v)$ contains most of the structural information in an image.
- Magnitude $|F(u, v)|$ primarily determines intensity and contrast.
- Swapping magnitude and phase between images confirmed that phase dominates perceived structure.

- **Spatial Transformations:**

- Circular shifting preserves $|F(u, v)|$ but introduces predictable phase shifts.
- Linear translation introduces a linear phase term in the frequency domain, changing the position of image features.

- **Filtering:**

- Filters selectively enhance or suppress frequency components to reduce noise or highlight details.
- **Ideal filters** have sharp cutoffs but can introduce ringing artifacts in the spatial domain.
- **Butterworth filters** smooth the transition with a tunable order n , balancing detail preservation and artifact reduction.
- **Gaussian filters** provide the smoothest frequency response, minimizing artifacts at the cost of slight loss in sharpness.
- Filtering noisy images demonstrated how frequency-selective filtering can recover the underlying signal while preserving key features.

- **Further Work:**

- Explore adaptive or anisotropic filtering techniques.
- Consider multi-resolution transforms, such as wavelets, for localized frequency analysis.
- Optimize filter parameters mathematically, e.g., by minimizing spatial-domain error metrics, for robust noise removal.

Table of Contents

Preamble	1
Loading the two new images and modifying for accuracy	1
Task 1	1
Task 2	3

Preamble

```
close all
clc
```

Loading the two new images and modifying for accuracy

```
img1 = imread("image1.png");
img1 = rgb2gray(img1); % convert to grayscale if RGB
img_1 = double(img1);

img2 = imread("image2.png");
img2 = rgb2gray(img2); % convert to grayscale if RGB
img2 = imresize(img2, size(img1));
img_2 = double(img2);

% Sizes of Images
[M_1, N_1] = size(img_1);
[M_2, N_2] = size(img_2);

%2023AAPS0013P -> A = 3, B = 3
A = 3;
B = 3;
```

Task 1

Part 1

```
fft_1 = fft2(img_1);
fftshift_1 = fftshift(fft_1);
mag_1 = log(1 + abs(fftshift_1));
phase_1 = angle(fftshift_1);

fft_2 = fft2(img_2);
fftshift_2 = fftshift(fft_2);
mag_2 = log(1 + abs(fftshift_2));
phase_2 = angle(fftshift_2);

figure();
```

```

subplot(2, 3, 1); imshow(img1); title("Greyscale Image 1")
subplot(2, 3, 2); imshow(mag_1, []); title("Log Magnitude Plot of Image 1")
subplot(2, 3, 3); imshow(phase_1, []); title("Phase Plot of Image 1")
subplot(2, 3, 4); imshow(img2); title("Greyscale Image 2");
subplot(2, 3, 5); imshow(mag_2, []); title("Log Magnitude Plot of Image 2")
subplot(2, 3, 6); imshow(phase_2, []); title("Phase Plot of Image 2")

```

% Part 3 - Swapping the magnitude and phase

```

mixed_1 = mag_1 .* exp(1i * phase_2);
mixed_1 = ifft2(ifftshift(mixed_1));
mixed_1 = mat2gray(real(mixed_1));

```

```

mixed_2 = mag_2 .* exp(1i * phase_1);
mixed_2 = ifft2(ifftshift(mixed_2));
mixed_2 = mat2gray(real(mixed_2));

```

```

figure();
subplot(1, 2, 1); imshow(mixed_1); title("Magnitude of 1 + Phase of 2");
subplot(1, 2, 2); imshow(mixed_2); title("Magnitude of 2 + Phase of 1");

```

% Part 4 - Shifting

```

shift_x = A * 10 + 5;
shift_y = B * 10 + 5;

```

```

img_1_shifted = circshift(img_1, [shift_y, shift_x]);
img_2_shifted = circshift(img_2, [shift_y, shift_x]);

```

```

fft_shifted_1 = fftshift(fft2(img_1));
mag_shifted_1 = log(1 + abs(fft_shifted_1));
phase_shifted_1 = angle(fft_shifted_1);
fft_shifted_2 = fftshift(fft2(img_2));
mag_shifted_2 = log(1 + abs(fft_shifted_2));
phase_shifted_2 = angle(fft_shifted_2);

```

```

figure ();
subplot(2, 3, 1); imshow(img1); title("Greyscale Image 1")
subplot(2, 3, 2); imshow(mag_1, []); title("Log Magnitude Plot of Image 1")
subplot(2, 3, 3); imshow(phase_1, []); title("Phase Plot of Image 1")
subplot(2, 3, 4); imshow(mat2gray(img_1_shifted)); title("Greyscale Shifted Image 1");
subplot(2, 3, 5); imshow(mag_shifted_1, []); title("Log Magnitude Plot of Shifted Image 1")
subplot(2, 3, 6); imshow(phase_shifted_1, []); title("Phase Plot Shifted of Image 1")

```

```

figure ();
subplot(2, 3, 1); imshow(img2); title("Greyscale Image 2")
subplot(2, 3, 2); imshow(mag_2, []); title("Log Magnitude Plot of Image 2")
subplot(2, 3, 3); imshow(phase_2, []); title("Phase Plot of Image 2")
subplot(2, 3, 4); imshow(mat2gray(img_2_shifted)); title("Greyscale Shifted Image 2");
subplot(2, 3, 5); imshow(mag_shifted_2, []); title("Log Magnitude Plot of Shifted Image 2")

```

```

subplot(2, 3, 6); imshow(phase_shifted_2, []); title("Phase Plot Shifted of
Image 2")

% Part 5 - Scaling and Resizing Image
img_1_compressed = imresize(img_1, 1/A);
img_1_expanded = imresize(img_1, B);

fft_compressed_1 = fftshift(fft2(img_1_compressed));
mag_compressed_1 = mat2gray(log(1 + abs(fft_compressed_1)));

fft_expanded_1 = fftshift(fft2(img_1_expanded));
mag_expanded_1 = mat2gray(log(1 + abs(fft_expanded_1)));

figure();
subplot(2, 3, 1); imshow(img1); title("Original Image");
subplot(2, 3, 2); imshow(mat2gray(img_1_compressed)); title("Compressed
Image");
subplot(2, 3, 3); imshow(mat2gray(img_1_expanded)); title("Expanded Image");
subplot(2, 3, 4); imshow(mag_1, []); title("Magnitude Plot of Original")
subplot(2, 3, 5); imshow(mag_compressed_1, []); title("Magnitude Plot of
Compressed");
subplot(2, 3, 6); imshow(mag_expanded_1, []); title("Magnitude Plot of
Expanded");

% Part 6 - Rotation of Image
angles = [0, 30, 45, 60, 90];
figure();
for k = 1:length(angles)
    theta = angles(k);

    img_rot = imrotate(img2, theta, 'bilinear', 'crop');

    F = fftshift(fft2(img_rot));
    mag = log(1 + abs(F));

    subplot(2, 5, k);
    imshow((img_rot));
    title(['Image after ', num2str(theta), '° rotation']);
    subplot(2, 5, k+5);
    imshow(mag, []);
    title(['Magnitude at ', num2str(theta), '° rotation']);
end

```

Task 2

LPF

```

radii = [20, 50, 100];
[x1, y1] = meshgrid(1:N_1, 1:M_1);
center1 = [ceil(N_1/2), ceil(M_1/2)];
dist1 = sqrt((x1-center1(1)).^2 + (y1-center1(2)).^2);

figure("Name", "Image 1: Circular LPF");

```

```

imgs1 = {img1};
F1s    = {fft_1};

for k = 1:length(radai)
    H = dist1 <= radai(k); % circular mask
    F1_filt = fft_shifted_1 .* H;
    img1_filt = real(ifft2(ifftshift(F1_filt)));

    imgs1{end+1} = img1_filt;
    F1s{end+1}    = F1_filt;
end

for k = 1:length(imgs1)
    % Row 1: Images
    subplot(3, length(imgs1), k);
    imshow(imgs1{k}, []);
    if k==1, title("Original"); else, title(['r = ', num2str(radai(k-1))]);
end

    % Row 2: Magnitude
    subplot(3, length(imgs1), length(imgs1)+k);
    imshow(log(1+abs(F1s{k})), []);
    title("Magnitude");

    % Row 3: Phase
    subplot(3, length(imgs1), 2*length(imgs1)+k);
    imshow(angle(F1s{k}), []);
    title("Phase");
end

rect_sizes = [20, 50, 100];
[x2, y2] = meshgrid(1:N_2, 1:M_2);
center2 = [ceil(N_2/2), ceil(M_2/2)];

figure("Name", "Image 2: Rectangular LPF");

imgs2 = {img2};
F2s    = {fft_2};

for k = 1:length(rect_sizes)
    rx = rect_sizes(k);
    ry = rect_sizes(k); % square filter
    H = (abs(x2-center2(1)) <= rx) & (abs(y2-center2(2)) <= ry);

    F2_filt = fft_shifted_2 .* H;
    img2_filt = real(ifft2(ifftshift(F2_filt)));

    imgs2{end+1} = img2_filt;
    F2s{end+1}    = F2_filt;
end

for k = 1:length(imgs2)
    % Row 1: Images

```

```

        subplot(3, length(imgs2), k);
        imshow(imgs2{k}, []);
        if k==1, title("Original"); else, title(['size = ',
num2str(rect_sizes(k-1))]); end

    % Row 2: Magnitude
    subplot(3, length(imgs2), length(imgs2)+k);
    imshow(log(1+abs(F2s{k})), []);
    title("Magnitude");

    % Row 3: Phase
    subplot(3, length(imgs2), 2*length(imgs2)+k);
    imshow(angle(F2s{k}), []);
    title("Phase");
end

% 2. Butterworth LPF (orders 1, 2, 4)
orders = [1,2,4];
D0 = 50; % cutoff radius
[x1,y1] = meshgrid(1:N_1, 1:M_1);
center1 = [ceil(N_1/2), ceil(M_1/2)];
D = sqrt((x1-center1(1)).^2 + (y1-center1(2)).^2);

figure("Name", "Image 1: Butterworth LPF");

imgs1 = {img1};
F1s    = {fft_1};

for n = orders
    H = 1 ./ (1 + (D./D0).^(2*n));
    F_filt = fft_shifted_1 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));

    imgs1{end+1} = img_filt;
    F1s{end+1}   = F_filt;
end

for k = 1:length(imgs1)
    subplot(3,length(imgs1),k), imshow(imgs1{k}, []);
    if k==1, title("Original"); else, title(['n = ', num2str(orders(k-1))]);
end
    subplot(3,length(imgs1),length(imgs1)+k), imshow(log(1+abs(F1s{k})), []),
title("Magnitude");
    subplot(3,length(imgs1),2*length(imgs1)+k), imshow(angle(F1s{k}), []),
title("Phase");
end

% 3. Gaussian LPF (D0=25,50,100)
cutoffs = [25,50,100];
figure("Name", "Image 1: Gaussian LPF");

imgs1 = {img1};
F1s    = {fft_1};

```

```

for D0 = cutoffs
    H = exp(-(D.^2)./(2*(D0^2)));
    F_filt = fft_shifted_1 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));

    imgs1{end+1} = img_filt;
    F1s{end+1} = F_filt;
end

for k = 1:length(imgs1)
    subplot(3,length(imgs1),k), imshow(imgs1{k},[]);
    if k==1, title("Original"); else, title(['D0 = ',num2str(cutoffs(k-1))]);
end
    subplot(3,length(imgs1),length(imgs1)+k), imshow(log(1+abs(F1s{k})),[]),
title("Magnitude");
    subplot(3,length(imgs1),2*length(imgs1)+k), imshow(angle(F1s{k}),[]),
title("Phase");
end

% 4. Ideal HPF (Disk + Rectangle)
radii = [20,50,100];
rect_sizes = [20,50,100];

figure("Name","Image 2: Ideal Circular HPF");
imgs1 = {img2}; F1s = {fft_2};
for r = radii
    H = double(D > r);
    F_filt = fft_shifted_2 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));
    imgs1{end+1} = img_filt; F1s{end+1} = F_filt;
end
for k=1:length(imgs1)
    subplot(3,length(imgs1),k), imshow(imgs1{k},[]);
    if k==1, title("Original"); else, title(['r = ',num2str(radii(k-1))]); end
    subplot(3,length(imgs1),length(imgs1)+k), imshow(log(1+abs(F1s{k})),[]),
title("Magnitude");
    subplot(3,length(imgs1),2*length(imgs1)+k), imshow(angle(F1s{k}),[]),
title("Phase");
end

[x2,y2] = meshgrid(1:N_2, 1:M_2);
center2 = [ceil(N_2/2), ceil(M_2/2)];
figure("Name","Image 2: Ideal Rectangular HPF");

imgs2 = {img2}; F2s = {fft_2};
for s = rect_sizes
    H = ~((abs(x2-center2(1))<=s) & (abs(y2-center2(2))<=s));
    F_filt = fft_shifted_2 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));
    imgs2{end+1} = img_filt; F2s{end+1} = F_filt;
end
for k=1:length(imgs2)

```

```

        subplot(3,length(imgs2),k), imshow(imgs2{k},[]);
        if k==1, title("Original"); else, title(['size = ',num2str(rect_sizes(k-1))]); end
        subplot(3,length(imgs2),length(imgs2)+k), imshow(log(1+abs(F2s{k})),[]),
title("Magnitude");
        subplot(3,length(imgs2),2*length(imgs2)+k), imshow(angle(F2s{k}),[]),
title("Phase");
end

% 5. Butterworth HPF (orders 1,2,4)
orders = [1,2,4]; D0 = 50;
figure("Name","Image 1: Butterworth HPF");
imgs1 = {img2}; F1s = {fft_2};
for n = orders
    H = 1 ./ (1 + (D0./D).^(2*n));
    H(D==0) = 0; % avoid div by 0
    F_filt = fft_shifted_2 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));
    imgs1{end+1} = img_filt; F1s{end+1} = F_filt;
end
for k=1:length(imgs1)
    subplot(3,length(imgs1),k), imshow(imgs1{k},[]);
    if k==1, title("Original"); else, title(['n = ',num2str(orders(k-1))]);
end
    subplot(3,length(imgs1),length(imgs1)+k), imshow(log(1+abs(F1s{k})),[]),
title("Magnitude");
    subplot(3,length(imgs1),2*length(imgs1)+k), imshow(angle(F1s{k}),[]),
title("Phase");
end

% 6. Gaussian HPF (D0=30,60,120)
cutoffs = [30,60,120];
figure("Name","Image 1: Gaussian HPF");
imgs1 = {img2}; F1s = {fft_2};
for D0 = cutoffs
    H = 1 - exp(-(D.^2)./(2*(D0^2)));
    F_filt = fft_shifted_2 .* H;
    img_filt = real(ifft2(ifftshift(F_filt)));
    imgs1{end+1} = img_filt; F1s{end+1} = F_filt;
end
for k=1:length(imgs1)
    subplot(3,length(imgs1),k), imshow(imgs1{k},[]);
    if k==1, title("Original"); else, title(['D0 = ',num2str(cutoffs(k-1))]);
end
    subplot(3,length(imgs1),length(imgs1)+k), imshow(log(1+abs(F1s{k})),[]),
title("Magnitude");
    subplot(3,length(imgs1),2*length(imgs1)+k), imshow(angle(F1s{k}),[]),
title("Phase");
end

% Part 7 & 8

```

```

f_noise = A*10 + 100;
W = 10;
BW = 15;
n_butter = 2;
noise_amp = 2000;

I1 = double(img_1);
I2 = double(img_2);

images = {I1, I2};
names = {'Image 1', 'Image 2'};

for idx = 1:2
    I = images{idx};
    [M, N] = size(I);

    [u, v] = meshgrid(1:N, 1:M);
    center = [ceil(N/2), ceil(M/2)];
    D = sqrt((u-center(1)).^2 + (v-center(2)).^2);

    [xgrid, ~] = meshgrid(1:N, 1:M);
    noise = noise_amp * sin(2*pi * f_noise * xgrid / N);
    I_noisy = I + noise;

    F_orig_shift = fftshift(fft2(I));
    F_noisy_shift = fftshift(fft2(I_noisy));

    D0 = f_noise;
    H_ideal = ones(M, N);
    H_ideal(abs(D - D0) <= (W/2)) = 0;
    F_ideal_shift = F_noisy_shift .* H_ideal;
    I_ideal = real(ifft2(ifftshift(F_ideal_shift)));

    arg = zeros(size(D));
    safeMask = denom ~= 0;
    arg(safeMask) = (D(safeMask) .* BW) ./ denom(safeMask);
    H_butt(~safeMask) = 0;
    F_butt_shift = F_noisy_shift .* H_butt;
    I_butt = real(ifft2(ifftshift(F_butt_shift)));
    imgs = {I, I_noisy, I_ideal, I_butt};
    Fshifts = {F_orig_shift, F_noisy_shift, F_ideal_shift, F_butt_shift};
    colTitles = {'Original', 'Noisy', 'Ideal BR', 'Butterworth BR'};

    figure('Name', [names{idx} ' - BR Comparison'], 'Position', [50 50 1300
600]);
    for c = 1:4
        subplot(3,4,c);
        imshow(mat2gray(imgs{c}), []);
        title(colTitles{c});
        subplot(3,4,4 + c);
        imagesc(log(1 + abs(Fshifts{c})));
        axis image off;
        title(['Magnitude - ' colTitles{c}]);

```

```

        colormap gray;
        ph = angle(Fshifts{c});
        imagesc(ph);
        axis image off;
        title(['Phase - ' colTitles{c}]);
        colormap gray;
    end
end

Error using imread>get_full_filename (line 635)
Unable to find file "image1.png".

Error in imread (line 395)
    fullname = get_full_filename(filename);
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Error in Assignment2 (line 6)
img1 = imread("image1.png");
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Published with MATLAB® R2025b